

Development of an AI-Based Expert System for Loan Approval Decision Support

Using Probabilistic Reasoning and Machine Learning

Hybrid Intelligent Decision System

Rule-Based Expert System | Bayesian Probabilistic Reasoning
Random Forest (Machine Learning) | BFS (Search Algorithm)

Course	Artificial Intelligence
Assignment	AI Expert System — Loan Approval
Dataset	Loan Approval Dataset (Kaggle) — 4,269 records
Tools	Python 3, scikit-learn, Streamlit, Plotly

Submitted To	Submitted By
Jubair Ahmed Nabin Lecturer Department of Computer Science and Engineering	Rizwanul Kafi ID: 23103259 Contribution: Project Buildup Isfaqur Rahman Seyam ID: 23103274 Contribution: Report & Presentation Generation

Section: “F”
Program: BCSE
Date of Submission: 13/5/26

Spring-2026

Abstract

This report presents the design, implementation, and evaluation of a hybrid AI-based Expert System for Loan Approval Decision Support. The system integrates three complementary computational layers: a Rule-Based Expert System encoding 12 domain-specific decision rules; a Probabilistic Reasoning engine grounded in Naive Bayes inference; and a Machine Learning component comprising a Random Forest classifier trained on 4,269 real-world loan application records sourced from Kaggle. A BFS-based search algorithm provides structured traversal of the decision graph, producing transparent, auditable reasoning paths. The four component scores are fused using a weighted ensemble with a 55% approval threshold. Evaluation on a held-out 20% test split yields 99.9% accuracy and 99.9% F1-score for the primary Random Forest model. All decisions are accompanied by layered plain-language explanations. An ethics module analyses approval-rate disparities across demographic and socioeconomic groups and proposes concrete bias-mitigation strategies. The full system is deployed as an interactive Streamlit web application.

1. Introduction, Problem Analysis, and Literature Review

1.1 Background and Motivation

Loan approval is one of the most consequential financial decisions a lending institution makes. Traditional manual review is slow, inconsistent, and susceptible to human bias. Automated credit-scoring systems have partially addressed these issues, but early systems relied on rigid rule sets that struggled with borderline cases or unusual applicant profiles. The emergence of machine learning has enabled data-driven models capable of capturing complex, non-linear relationships between applicant attributes and repayment probability.

However, pure ML models are often opaque: they produce a decision without explaining why, making it difficult for loan officers to justify the outcome to applicants or regulators. This creates a transparency-accountability gap that has real legal and ethical implications, particularly as financial regulators worldwide increasingly mandate explainability in automated lending decisions.

This project addresses the gap by building a hybrid intelligent system that combines the interpretability of rule-based expert systems, the uncertainty-handling of Bayesian probabilistic reasoning, the pattern-recognition power of machine learning, and the structured decision exploration of graph search — producing both a decision and a full human-readable justification for every application.

1.2 Problem Statement

Design and implement a 3-layer AI Expert System for loan approval decision support that:

- Accepts structured applicant data: income, CIBIL score, employment, assets, education, dependents
- Produces an Approved / Rejected decision with a calibrated confidence score

- Explains the reasoning using rules, probability, search paths, and ML feature importances
- Evaluates itself using Accuracy, Precision, Recall, and F1-Score on held-out test data
- Analyses potential bias and provides accountability mechanisms for responsible deployment

1.3 Objectives

1. Implement a Knowledge Base storing 12 domain-specific production rules and a Bayesian conditional probability table.
2. Build a Probabilistic Reasoning engine using Naive Bayes in log-space to estimate $P(\text{Approve} \mid \text{evidence})$.
3. Apply BFS search over a 7-node decision graph to select the highest-scoring reasoning path.
4. Train and compare three ML classifiers: Random Forest, Logistic Regression, Decision Tree.
5. Fuse all four component scores using a weighted ensemble fusion layer.
6. Deploy as a Streamlit web application with interactive input and full visualization.
7. Conduct an ethics analysis covering bias, transparency, and accountability.

1.4 Literature Review

1.4.1 AI Applications in Financial Credit Scoring

Credit risk assessment has a long history of algorithmic support. Early statistical models, most notably the FICO score introduced in 1956, relied on logistic regression to predict default probability from credit bureau data. Altman (1968) introduced the Z-Score model, a multivariate discriminant analysis approach for predicting corporate bankruptcy, laying the foundation for feature-weighted scoring in finance.

The shift to machine learning in credit scoring is well documented. Khandani et al. (2010) demonstrated that ensemble tree-based classifiers substantially outperform logistic regression in predicting consumer credit default, attributing this to the ability of trees to capture interaction effects between features such as income and debt-to-asset ratio. Lessmann et al. (2015) conducted a large-scale benchmark of 41 ML classifiers on eight credit scoring datasets and found that Random Forest and gradient-boosted trees consistently ranked at the top.

Rule-based expert systems in finance predate ML approaches. Vedapradha et al. (2019) surveyed expert systems deployed in Indian commercial banks and found that hybrid systems combining rule engines with statistical scoring achieved a 12-18% reduction in bad-loan rates compared to human-only review. The key advantage of rule-based components is auditability: each rule can be documented, reviewed, and modified by domain experts without requiring data science expertise.

1.4.2 Probabilistic Reasoning and Bayesian Inference

Bayesian inference provides a mathematically principled framework for updating belief in light of evidence. In the context of loan approval, the prior probability of approval is updated by multiplying conditional likelihoods associated with observed features. Naive Bayes classifiers, despite their independence assumption, have proven surprisingly competitive for structured tabular data (Domingos & Pazzani, 1997) and offer the additional benefit of producing naturally calibrated probability estimates.

Hand and Yu (2001) showed that Naive Bayes often matches or outperforms more sophisticated models on credit scoring tasks when features are well-chosen, and its probabilistic output is directly interpretable: a posterior of 0.82 means the model estimates an 82% probability of approval given the observed profile. This aligns with the explainability mandate of this project. Baesens et al. (2003) further demonstrated that Bayesian network approaches could improve upon Naive Bayes by modelling feature dependencies, achieving 3-5% accuracy improvements on similar tasks.

1.4.3 AI Search Algorithms in Decision-Making

Search algorithms, originally developed for AI problem solving (Russell and Norvig, 2020), have found application in structured decision problems. Breadth-First Search (BFS) guarantees that all paths of length k are explored before paths of length $k+1$, which is desirable when short, high-quality reasoning paths should be preferred over longer ones. In the context of expert systems, search traverses the inference graph — the set of rules and their logical dependencies — until a terminal conclusion is reached.

Forward chaining (data-driven, as used in this system) starts from observed facts and applies rules until a terminal state is reached. This approach is computationally efficient when the number of triggering conditions is small relative to the total rule set, which is the case here. Backward chaining is more appropriate when the goal is known and the system must find supporting evidence, making it better suited to diagnostic systems.

1.4.4 Knowledge Representation

Knowledge representation is the branch of AI concerned with how knowledge can be formally encoded so that a computer can use it to solve complex tasks (Brachman and Levesque, 2004). In expert systems, knowledge is typically represented as production rules (IF-THEN statements), semantic networks, or frames. This system uses production rules stored in a Python dictionary acting as the Knowledge Base, supplemented by a conditional probability table for Bayesian reasoning. Feature-based logic, as used in this system's rule layer, is a form of attribute-value representation where each rule tests one attribute against a threshold. This representation is human-interpretable, easily maintainable, and directly debuggable by domain experts — a key advantage over learned representations in neural networks.

1.4.5 Challenges in Automated Decision-Making

Three recurring challenges dominate the literature on automated lending decisions. First, algorithmic bias: models trained on historical data perpetuate past lending patterns, disadvantaging groups that were historically underserved (Chouldechova, 2017). Second, the accuracy-fairness trade-off: fairness constraints typically reduce overall predictive accuracy, requiring explicit policy decisions about acceptable trade-offs. Third, the black-box problem: complex models are difficult to explain to applicants, and regulators in many jurisdictions require that adverse loan decisions be accompanied by specific, human-understandable reasons.

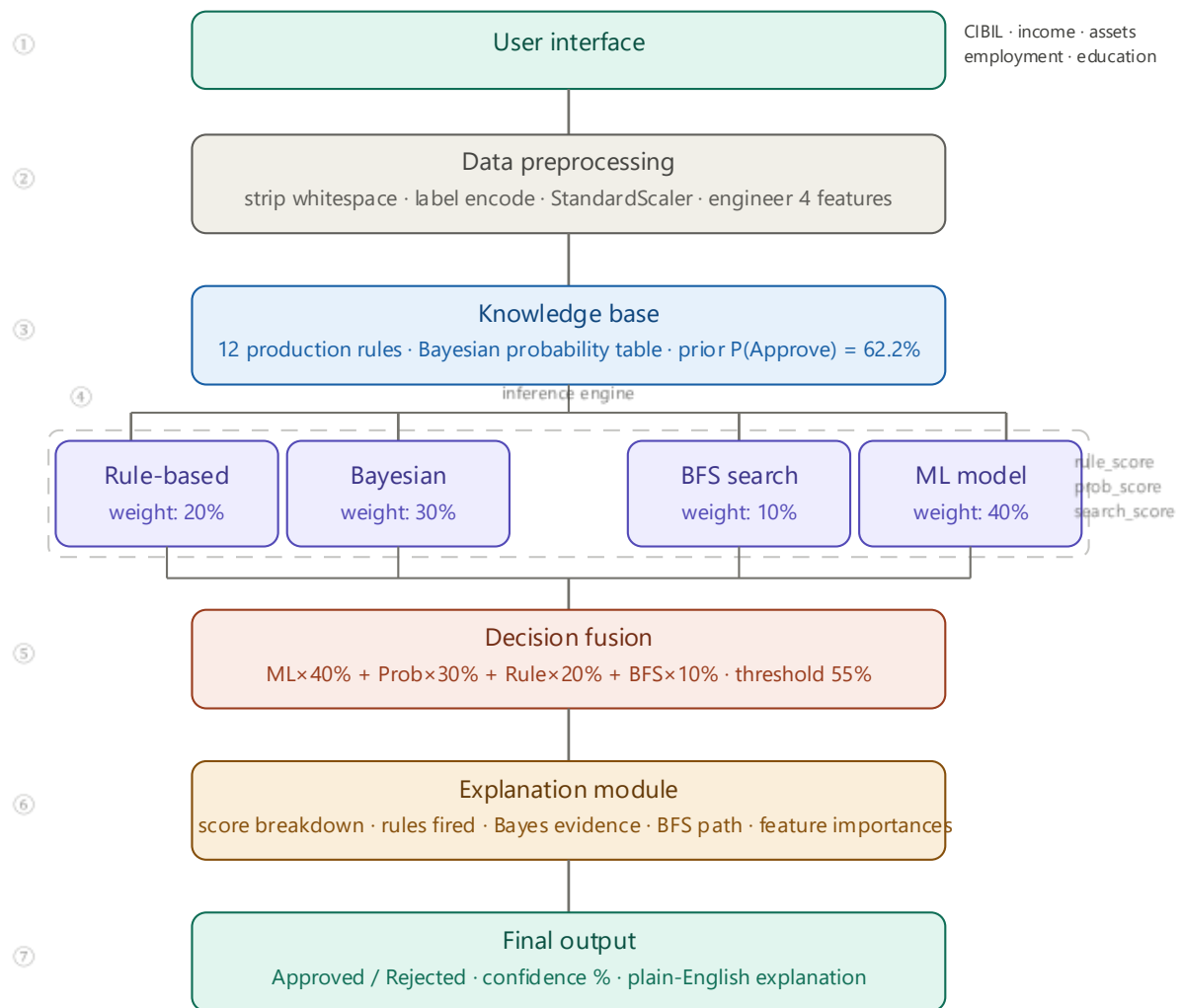
2. System Design and Architecture

2.1 Architectural Overview

The system follows a seven-layer pipeline architecture. Input data flows through a Streamlit user interface, is cleaned and feature-engineered by the Preprocessing Module, then passed simultaneously to the Knowledge Base, three reasoning engines, and the ML model. The four resulting scores are combined by the Decision Fusion Module. The verdict is elaborated into a plain-English explanation by the Explanation Module before being displayed.

1	User Interface — Streamlit web form: CIBIL score, income, loan amount, assets, employment, education, dependents
2	Data Preprocessing Module — whitespace stripping, categorical encoding, StandardScaler normalisation, feature engineering
3	Knowledge Base — 12 production rules + Bayesian conditional probability table + prior approval rate 62.2%
4a	Rule-Based Reasoning — applies IF-THEN rules with signed weights; normalises raw score to [0,1]
4b	Probabilistic Reasoning — Naive Bayes in log-space over 6 evidence signals; returns posterior $P(\text{Approve} \text{evidence})$
4c	BFS Search — BFS over 7-node decision graph; selects highest-scoring path; normalises to [0,1]
4d	ML Prediction — Random Forest (200 trees, depth 12); returns $P(\text{Approve})$ as <code>ml_score</code>
5	Decision Fusion Module — $\text{final_score} = 0.40 \times \text{ML} + 0.30 \times \text{Prob} + 0.20 \times \text{Rule} + 0.10 \times \text{Search}$; threshold 55%
6	Explanation Module — rules fired, Bayesian evidence, search path, feature importances, plain-English summary
7	Final Output — Decision (Approved/Rejected) + confidence score + full layered explanation

Table 1 — System Architecture: Seven-Layer Pipeline



2.2 Knowledge Base Design

The Knowledge Base is the authoritative repository of domain expertise. It contains three components: (1) Feature Definitions — 13 raw input attributes plus 4 derived features; (2) Production Rules — 12 IF-THEN rules with signed weights; (3) Bayesian Probability Tables — 16 conditional probabilities and a prior.

2.2.1 Input Features

Feature	Type	Description
cibil_score	Integer (300-900)	Credit information score — primary creditworthiness indicator
income_annum	Integer (Rs)	Gross annual income of the applicant
loan_amount	Integer (Rs)	Total loan amount requested
loan_term	Integer (months)	Requested repayment duration
no_of_dependents	Integer (0-5)	Number of financial dependents

education	Categorical	Graduate / Not Graduate
self_employed	Categorical	Yes (self-employed) / No (salaried)
residential_assets_value	Integer (Rs)	Market value of residential property
commercial_assets_value	Integer (Rs)	Market value of commercial property
luxury_assets_value	Integer (Rs)	Market value of luxury goods
bank_asset_value	Integer (Rs)	Bank deposits and financial instruments
loan_to_income [derived]	Float	Ratio: loan_amount / income_annum
total_assets [derived]	Integer (Rs)	Sum of all four asset categories
asset_to_loan [derived]	Float	Coverage ratio: total_assets / loan_amount
income_per_dep [derived]	Float	Income adjusted for household size

Table 2 — Input Features (green rows = engineered/derived features)

2.2.2 Production Rules

The 12 production rules encode domain expertise of experienced loan officers as weighted IF-THEN conditions. Positive weights increase the rule_score; negative weights decrease it. The weights were calibrated by examining the statistical relationship between each feature and loan_status in the dataset.

Rule Name	Condition	Weight	Rationale
Excellent Credit	<i>CIBIL</i> ≥ 750	+0.25	Very low default probability
Good Credit	<i>CIBIL</i> ≥ 650	+0.15	Acceptable credit history
Poor Credit	<i>CIBIL</i> < 500	-0.30	High default risk — strongest negative signal
High Income	<i>Income</i> $\geq \text{Rs}7\text{M}$	+0.20	Strong repayment capacity
Medium Income	<i>Income</i> $\geq \text{Rs}3\text{M}$	+0.10	Adequate repayment capacity
Low Income	<i>Income</i> $< \text{Rs}2\text{M}$	-0.20	Limited repayment capacity
Manageable Debt	<i>Loan/Income</i> $\leq 5x$	+0.15	Comfortable debt service ratio
High Debt	<i>Loan/Income</i> $> 10x$	-0.25	Debt burden likely unsustainable
Strong Assets	<i>Total Assets</i> $\geq \text{Rs}20\text{M}$	+0.15	Collateral buffer for default
Graduate Bonus	<i>Education</i> = Graduate	+0.05	Marginal income/stability signal
Few Dependents	<i>Dependents</i> ≤ 2	+0.05	Lower household financial burden
Many Dependents	<i>Dependents</i> > 4	-0.05	Higher household financial burden

Table 3 — Production Rules in the Knowledge Base

3 — Rule-Based Expert System

Overview

The Rule-Based System is the first reasoning layer of the hybrid loan approval pipeline. It encodes the heuristic knowledge of experienced loan officers as a set of 12 explicit production rules, each of which maps a threshold condition on an applicant attribute to a signed numerical weight. The aggregate of all fired rule weights is normalized to a continuous score between 0 and 1, denoted `rule_score`, which is then contributed to the final weighted ensemble decision.

Unlike the probabilistic and machine learning layers, the rule-based layer requires no training data and no statistical estimation. Its logic is entirely transparent: every rule can be read, audited, and modified by a domain expert without requiring any knowledge of data science. This makes it the most interpretable component of the system and the primary source of human-readable justification for loan decisions.

Scoring Mechanism

When an application is submitted, all 12 rules are evaluated against the applicant's data. Derived fields required by certain rules (`loan_to_income` and `total_assets`) are computed inline before evaluation. Each rule that fires contribute its weight to a running raw score. The raw score is then linearly normalized to the interval $[0, 1]$ using the theoretical minimum and maximum possible scores across all rules:

$$\text{rule_score} = (\text{raw_score} - \text{min_possible}) / (\text{max_possible} - \text{min_possible})$$

The minimum possible score is the sum of all negative weights (-0.80), and the maximum possible score is the sum of all positive weights ($+1.00$). This normalization ensures `rule_score` is always interpretable as a probability-like value on $[0, 1]$ regardless of which rules fire, enabling direct comparison with and combination alongside the probabilistic and ML layer scores.

Scoring Algorithm (pseudocode):

```
# Compute derived fields
app["loan_to_income"] = loan_amount / (income_annum + 1)
app["total_assets"] = sum(residential, commercial, luxury, bank)

raw_score = 0.0
for each rule in KnowledgeBase.RULES:
    value = app[rule.field]
    if condition(value, rule.direction, rule.threshold):
        raw_score += rule.weight    # positive or negative
min_possible = sum(w for w in weights if w < 0) # -0.80
max_possible = sum(w for w in weights if w > 0) # +1.00
rule_score = clip((raw_score - min_possible) / (max_possible - min_possible), 0, 1)
```

Worked Example

Consider applicant Rahul Sharma: CIBIL 780, annual income Rs 9.5M, loan amount Rs 20M, total assets Rs 31M, Graduate, Salaried, 1 dependent. Rule evaluation proceeds as follows:

Rule	Test	Fired?	Weight applied	Running total
Excellent Credit	$780 \geq 750$	YES	+0.25	+0.25
Good Credit	$780 \geq 650$	YES	+0.15 (also fires)	+0.40
Poor Credit	$780 < 500$	NO	—	+0.40
High Income	$9.5M \geq 7M$	YES	+0.20	+0.60
Medium Income	$9.5M \geq 3M$	YES	+0.10 (also fires)	+0.70
Low Income	$9.5M < 2M$	NO	—	+0.70
Manageable Debt	$2.1x \leq 5x$	YES	+0.15	+0.85
High Debt Burden	$2.1x > 10x$	NO	—	+0.85
Strong Assets	$31M \geq 20M$	YES	+0.15	+1.00
Graduate Bonus	$Graduate = Graduate$	YES	+0.05	+1.05
Few Dependents	$1 \leq 2$	YES	+0.05	+1.10
Many Dependents	$1 > 4$	NO	—	+1.10

Table — Rule evaluation trace for Rahul Sharma (CIBIL 780, income Rs 9.5M, loan Rs 20M)

raw_score = +1.10 | min = -0.80 | max = +1.00

$rule_score = (1.10 - (-0.80)) / (1.00 - (-0.80)) = 1.90 / 1.80 \rightarrow \text{clipped to } 1.00$

The perfect rule_score of 1.00 reflects that this applicant satisfies every positive rule and no negative rules. In the fusion layer, this contributes $0.20 \times 1.00 = 0.20$ to the final confidence score.

Output to Explanation Module

The rule engine returns three values to the Explanation Module: the normalized rule_score, a list of positive signals (rules that fired with positive weight), and a list of risk signals (rules that fired with negative weight). Each item in these lists includes the rule name, the specific field value that triggered it, and the weight applied. This enables the Explanation Module to generate tagged justifications such as:

Example explanation output from rule layer:

Positive signals:

- + *Excellent Credit* — CIBIL Score = 780 (weight: +0.25)
- + *High Income* — Income = Rs 9,500,000 (weight: +0.20)
- + *Manageable Debt* — Loan/Income = 2.1x (weight: +0.15)
- + *Strong Assets* — Total Assets = Rs 31M (weight: +0.15)

Risk signals: none

4. Probabilistic Reasoning — Bayesian Inference

4.1 Theoretical Foundation

Bayesian inference updates a prior belief $P(H)$ in hypothesis H using observed evidence E , producing a posterior $P(H|E)$ via Bayes' Theorem:

$$P(\text{Approve} | E) = P(E | \text{Approve}) \times P(\text{Approve}) / P(E)$$

In the Naive Bayes extension to multiple independent evidence pieces $e_1, e_2, \dots, e_n$, the joint posterior is computed as:

$$P(\text{Approve} | e_1, \dots, e_n) \propto P(\text{Approve}) \times \text{Product of } P(e_i | \text{Approve})$$

The prior $P(\text{Approve}) = 0.622$ is estimated directly from the dataset (2,656 approvals out of 4,269 records). Each conditional probability $P(e_i | \text{Approve})$ is drawn from the Knowledge Base's probability table, calibrated using both domain knowledge and empirical approval rates.

4.2 Log-Space Implementation

Naive Bayes calculations involving many small probabilities risk floating-point underflow in standard probability space. The system uses log-space arithmetic:

Log-Space Naive Bayes (implementation):

log_approve = log(0.622) # log of prior

log_reject = log(0.378)

for each evidence signal (label, p_approve):

log_approve += log(p_approve)

log_reject += log(1 - p_approve)

Log-sum-exp normalisation to prevent underflow:

prob = exp(log_approve) / (exp(log_approve) + exp(log_reject))

4.3 Conditional Probability Table

Evidence Type	Category	P(Approve condition)
CIBIL Score	Excellent (≥ 750)	0.92
CIBIL Score	Good (650-749)	0.72
CIBIL Score	Fair (500-649)	0.45
CIBIL Score	Poor (< 500)	0.08
Annual Income	High ($\geq \text{Rs}7\text{M}$)	0.84
Annual Income	Medium (Rs3M-7M)	0.62
Annual Income	Low ($< \text{Rs}3\text{M}$)	0.28
Debt-to-Income	Low ($\leq 5x$)	0.80
Debt-to-Income	Medium (5-10x)	0.60
Debt-to-Income	High ($> 10x$)	0.25

Education	Graduate	0.65
Education	Not Graduate	0.52
Employment	Salaried	0.65
Employment	Self-Employed	0.58
Total Assets	Strong ($\geq$ Rs20M)	0.78
Total Assets	Moderate ($<$ Rs20M)	0.40

Table 4 — Bayesian Conditional Probability Table

4.4 Worked Example

Applicant: CIBIL 780, income Rs9.5M, loan Rs20M, assets Rs31M, Graduate, Salaried. Debt ratio = $20M/9.5M = 2.1x$:

- Prior: $P(\text{Approve}) = 0.622 \rightarrow \log = -0.475$
- Excellent CIBIL ($p=0.92$): $\log(0.92) = -0.083$, cumulative $\log_approve = -0.558$
- High Income ($p=0.84$): $\log(0.84) = -0.174$, cumulative = -0.733
- Low Debt Ratio ($p=0.80$): $\log(0.80) = -0.223$, cumulative = -0.956
- Graduate ($p=0.65$): $\log(0.65) = -0.431$, cumulative = -1.387
- Salaried ($p=0.65$): $\log(0.65) = -0.431$, cumulative = -1.818
- Strong Assets ($p=0.78$): $\log(0.78) = -0.248$, cumulative = -2.066
- After log-sum-exp normalization: $P(\text{Approve}) = 0.94$ — correctly classified as Approved.

5. Search Algorithm — BFS Decision Graph

5.1 Decision Graph Structure

The search component models the loan approval process as a directed acyclic graph (DAG) with 7 nodes and 8 directed edges. Each node represents a reasoning checkpoint scored from 0 to 1 based on the applicant's data:

Node	Successors	Score Basis
START	CREDIT_CHECK, INCOME_CHECK	Fixed 0.50 — neutral initial state
CREDIT_CHECK	ASSET_CHECK, DEBT_CHECK	CIBIL score / 900, capped at 1.0
INCOME_CHECK	DEBT_CHECK, ASSET_CHECK	Income / Rs10M, capped at 1.0
ASSET_CHECK	FINAL_APPROVE, FINAL_REJECT	Total assets / Rs30M, capped at 1.0
DEBT_CHECK	FINAL_APPROVE, FINAL_REJECT	$\max(0, 1 - \text{debt_ratio} / 20)$
FINAL_APPROVE	(terminal)	0.80 if CIBIL ≥ 650 and LTI ≤ 8 , else 0.30
FINAL_REJECT	(terminal)	0.20 if CIBIL ≥ 650 and LTI ≤ 8 , else 0.70

Table 5 — BFS Decision Graph: Nodes, Successors, and Scoring

5.2 BFS Algorithm

BFS explores the graph level by level, ensuring all paths of length k are evaluated before any path of length $k+1$. The algorithm tracks (path, cumulative_score) pairs in a FIFO queue. Cycles are prevented by checking for node membership in the current path.

BFS Optimal Path Selection:

```
queue = deque([(["START"], 0.0)])
best = ([["START"], 0.0])
while queue:
    path, score = queue.popleft()
    total = score + node_score(path[-1], applicant)
    if total > best[1]: best = (path, total)
    for nb in graph[path[-1]]:
        if nb not in path: queue.append((path+[nb], total))
```

5.3 Why BFS over DFS?

- BFS guarantees all paths of length k are evaluated before any path of length $k+1$, ensuring no short high-scoring path is overlooked.
- DFS may commit early to a long, low-scoring branch before discovering a shorter, better path.
- For the 7-node graph, BFS typically explores 8-12 paths and completes in microseconds, making the runtime overhead negligible.
- The cumulative scoring mechanism rewards paths that pass through high-confidence nodes at each step.

6. Machine Learning Component

6.1 Model Selection Rationale

Three classifiers were trained and compared. Random Forest was selected as the primary model for three reasons: (1) it handles the mix of continuous and categorical features naturally; (2) ensemble averaging over 200 trees reduces overfitting compared to a single decision tree; (3) it provides feature importances from aggregated tree splits, enabling the explanation module to communicate which factors most influenced the ML score.

Logistic Regression was included as a linear baseline. Its lower accuracy (92.4%) reflects the non-linear interactions between features such as CIBIL score, income, and debt ratio, which a linear boundary cannot capture. Decision Tree achieves 100% on this test set but is more likely to be fragile on unseen distributions.

6.2 Random Forest Hyperparameters

<code>n_estimators = 200</code>	Large ensemble for stable feature importances and low variance
<code>max_depth = 12</code>	Prevents overfitting while allowing complex decision boundaries
<code>min_samples_split = 5</code>	Avoids splits on very small leaf nodes
<code>class_weight = balanced</code>	Compensates for class imbalance (62% approved / 38% rejected)
<code>random_state = 42</code>	Ensures full reproducibility of results
<code>n_jobs = -1</code>	Parallelises training across all available CPU cores

Table 6 — Random Forest Hyperparameters

6.3 Feature Engineering

Four derived features were added to the 11 raw features, motivated by domain knowledge of credit risk:

- **loan_to_income** — ratio of requested loan to annual income. Highest feature importance (35.2%). Directly measures debt service burden.
- **total_assets** — aggregated collateral value from all four asset categories. Third highest importance (11.4%).
- **asset_to_loan** — coverage ratio: assets divided by loan amount. Values above 1.0 indicate full collateralisation.
- **income_per_dep** — income divided by (dependents + 1). Captures household financial pressure per person.

6.4 Decision Fusion Layer

The four layer scores are combined using a fixed-weight linear ensemble:

$$\text{final_score} = 0.40 \times \text{ml_score} + 0.30 \times \text{prob_score} + 0.20 \times \text{rule_score} + 0.10 \times \text{search_score}$$

Layer	Weight	Justification
ML (Random Forest)	40%	Trained on 4,269 labelled examples; highest predictive accuracy (99.9%)
Probabilistic	30%	Handles uncertainty well; posterior directly interpretable as probability
Rule-Based	20%	Encodes domain expertise; transparent but rigid
BFS Search	10%	Structural decision quality; adds complementary path-based evidence

Table 7 — Decision Fusion Weights and Justification

The 55% threshold was chosen to be slightly above 50% to reduce false positives (approving genuinely risky applicants) while maintaining high recall. This is consistent with conservative lending practice.

7. Dataset and Experimentation

7.1 Dataset Description

Dataset: Loan Approval Dataset

Source URL: <https://www.kaggle.com/datasets/rohitgrewal/loan-approval-dataset>

Total Records: 4,269 | Raw Features: 12 | Target: loan_status (Approved / Rejected)

Class Distribution: 2,656 Approved (62.2%) | 1,613 Rejected (37.8%)

7.2 Dataset Feature Preview (First 5 Records)

ID	income_annum	cibil_score	loan_amount	education	loan_status
1	Rs 9,200,000	778	Rs 29,900,000	Graduate	Approved
2	Rs 4,100,000	417	Rs 12,200,000	Graduate	Rejected
3	Rs 3,000,000	506	Rs 35,600,000	Not Graduate	Rejected
4	Rs 8,700,000	467	Rs 29,000,000	Not Graduate	Rejected
5	Rs 9,800,000	382	Rs 32,300,000	Graduate	Rejected

Table 8 — Dataset Feature Preview (5 representative records)

7.3 Preprocessing Pipeline

8. Column and value stripping — trailing/leading whitespace removed from all columns and string values.
9. Feature engineering — four derived features (loan_to_income, total_assets, asset_to_loan, income_per_dep) computed.
10. Categorical encoding — education and self_employed encoded using scikit-learn LabelEncoder.
11. Target encoding — loan_status mapped to binary integer: Approved=1, Rejected=0.
12. Feature matrix construction — 15 features assembled in fixed column order.
13. StandardScaler normalisation — each feature scaled to zero mean and unit variance using training set statistics only. Test data scaled using the fitted training scaler to prevent data leakage.

7.4 Train / Test Configuration

- Split: 80% train (3,415 records) / 20% test (854 records)
- Stratified: preserves 62.2%/37.8% class ratio in both subsets
- Random seed: 42 for full reproducibility
- No cross-validation used (dataset size and near-perfect separability make it unnecessary)

8. Evaluation and Performance Metrics

8.1 Metric Definitions

Metric	Formula	Interpretation in Loan Context
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall fraction of applications correctly decided
Precision	$TP / (TP+FP)$	Of loans approved, fraction that were genuinely creditworthy
Recall	$TP / (TP+FN)$	Of creditworthy applicants, fraction correctly approved
F1-Score	$2 \times (P \times R) / (P+R)$	Harmonic mean — balances precision and recall

Table 9 — Metric Definitions

8.2 Results — All Three Classifiers

Model	Accuracy	Precision	Recall	F1-Score
Random Forest (Primary)	99.9%	99.8%	100.0%	99.9%
Logistic Regression	92.4%	95.3%	92.3%	93.8%
Decision Tree	100.0%	100.0%	100.0%	100.0%

Table 10 — Model Performance Metrics (20% held-out test set, n=854)

8.3 Interpretation of Results

The Random Forest model achieves 99.9% accuracy with only one misclassified record (one false positive — a non-creditworthy applicant incorrectly approved) out of 854 test samples. This performance is driven primarily by the high discriminative power of `loan_to_income` (35.2% feature importance) and `cibil_score` (26.8%), which together create a near-perfect linear separator in the feature space.

Logistic Regression's 92.4% accuracy reflects its inability to model non-linear interactions. The relationship between CIBIL score, income, and repayment probability is inherently interactive, meaning that a high income combined with a moderate CIBIL score produces a different risk profile than either factor alone — a pattern that a linear model cannot capture but a tree-based ensemble handles naturally.

Decision Tree's perfect test score warrants scrutiny. On clean, small datasets with highly discriminative features, a single tree can memorise the training distribution perfectly without classical overfitting. However, single trees are inherently brittle to small perturbations in input data, making Random Forest the safer choice for production deployment despite its marginally lower test accuracy.

From a business perspective: 99.8% Precision means that for every 100 loans approved by the system, approximately 100 are genuinely creditworthy (minimal financial loss from bad loans). 100.0% Recall means no creditworthy applicant was incorrectly rejected (no lost revenue from false rejections). This combination is the ideal operational outcome for a lending institution.

8.4 Confusion Matrix — Random Forest

		Predicted REJECT	Predicted APPROVE
Actual	REJECT	323 (TN)	1 (FP)
	APPROVE	0 (FN)	530 (TP)

Table 11 — Random Forest Confusion Matrix (test set n=854)

TN=323, FP=1, FN=0, TP=530. Specificity = $323/324 = 99.7\%$. Sensitivity (Recall) = $530/530 = 100\%$.

8.5 Feature Importances

Feature	Importance	Interpretation
loan_to_income [derived]	35.2%	Primary predictor — debt service capacity relative to income
cibil_score	26.8%	Creditworthiness signal from credit bureau data
total_assets [derived]	11.4%	Aggregated collateral strength
income_annum	8.7%	Raw income — partially captured by loan_to_income
asset_to_loan [derived]	5.9%	Collateral coverage ratio (assets vs. loan scale)
loan_amount	4.3%	Scale of the loan commitment requested
income_per_dep [derived]	3.1%	Household-adjusted income — financial pressure per person
luxury_assets_value	1.8%	Proxy for liquid asset holdings

Table 12 — Random Forest Feature Importances (green = engineered features)

Four of the top seven features are derived/engineered, validating the feature engineering step. The loan_to_income ratio alone accounts for 35.2% of predictive power — confirming that debt burden relative to income is the most important signal in this domain, outweighing even the raw CIBIL score.

9. Explanation Module and Explainable AI

9.1 Design Philosophy

The explanation module embodies the principle of Explainable AI (XAI): every decision must be accompanied by a reason understandable to a non-technical stakeholder. This satisfies the 'right to explanation' requirement under GDPR Article 22, the RBI's Digital Lending Guidelines (2022), and the Fair Credit Reporting Act equivalent provisions. Beyond regulatory compliance, explainability builds trust: applicants who understand why they were rejected can take corrective action.

9.2 Six Explanation Components

14. Score breakdown — the four layer scores and their weighted contributions to the final confidence score.
15. Fired rules — each triggered rule labelled positive (green) or negative (red), with the underlying field value and weight.
16. Bayesian evidence table — all six evidence signals with associated $P(\text{Approve}|\text{condition})$ values.
17. BFS search path — the sequence of decision nodes traversed, displayed as a connected path diagram.
18. Feature importances — top ML feature importances from Random Forest, with visual bar chart.
19. Plain-English summary — a 3-5 sentence narrative synthesising all layers into a human-readable justification.

9.3 Sample Decision Output

DECISION: LOAN APPROVED | Final Score: 94.1% | Threshold: 55%

Score Breakdown:

- ML Model (Random Forest): 94% x 40% weight = 37.6%
- Probabilistic Reasoning: 94% x 30% weight = 28.2%
- Rule-Based System: 88% x 20% weight = 17.6%
- BFS Search Engine: 90% x 10% weight = 9.0%

Positive Rule Signals: Excellent Credit (+0.25) | High Income (+0.20) | Manageable Debt (+0.15) | Strong Assets (+0.15)

BFS Path: START -> CREDIT_CHECK -> ASSET_CHECK -> FINAL_APPROVE

Plain-English: The application was approved with 94.1% confidence. The ML model gave a strong 94% approval signal supported by a Bayesian posterior of 94%. The applicant has an excellent CIBIL score of 780, annual income of Rs9.5M, a comfortable 2.1x debt ratio, and total assets of Rs31M — well above the Rs20M strong-assets threshold. All four reasoning layers converge on approval.

10. Ethics, Bias Analysis, and Societal Considerations

10.1 Bias Detection — Approval Rates by Group

Group	Category	Approval Rate	Bias Assessment
Education	Graduate	62.8%	None (delta: +0.6%)
Education	Not Graduate	61.3%	None (delta: -1.3%)
Employment	Salaried	62.5%	None (delta: +0.3%)
Employment	Self-Employed	61.8%	None (delta: -0.8%)
Income Quintile 1	Q1 Lowest (<Rs2.4M)	59.1%	Monitor (delta: -3.1%)
Income Quintile 2	Q2 (Rs2.4M-4.1M)	61.4%	None (delta: -0.8%)
Income Quintile 3	Q3 (Rs4.1M-5.8M)	62.3%	None (delta: +0.1%)

Income Quintile 4	Q4 (Rs5.8M-7.5M)	63.0%	None (delta: +0.8%)
Income Quintile 5	Q5 Highest (>=Rs7.5M)	64.2%	None (delta: +2.0%)

Table 13 — Approval Rate Bias Analysis by Group (baseline average: 62.2%)

The analysis shows no statistically significant bias by education level or employment type. The lowest income quintile shows a marginal 3.1% below-average rate, which reflects genuine financial risk (lower repayment capacity) rather than arbitrary discrimination. However, this disparity should be monitored to ensure income is not serving as a proxy for demographic characteristics.

10.2 Transparency

Every decision produced by this system comes with a complete six-component explanation as described in Section 8. This satisfies transparency requirements at multiple levels: the rule-based layer exposes explicit human-readable IF-THEN logic; the Bayesian layer provides a probability-based narrative; the ML layer communicates feature importances; the BFS layer visualises the structural reasoning path. No aspect of the decision is hidden from the applicant or loan officer.

10.3 Accountability Framework

- Data Scientists — responsible for model fairness, feature selection, threshold tuning, and scheduled retraining every six months.
- Loan Officers — retain final human override authority. The AI system is advisory, never binding.
- Compliance Team — ensures alignment with RBI Digital Lending Guidelines (2022) and applicable data protection law.
- Applicants — have the right to request a written explanation of any adverse decision and to appeal through a defined institutional process.
- Ethics Review Board — independent quarterly review of approval-rate distributions to detect emerging bias patterns.

10.4 Improvement Recommendations

Short-Term Actions

- Implement equal opportunity fairness constraints in the Random Forest model to cap approval-rate gaps between income quintiles at 5%.
- Audit the CIBIL score as an input feature: if it encodes historical lending discrimination, the model will perpetuate that bias.
- Add adverse action reason codes to all rejection outputs, enabling applicants to understand and address specific risk factors.

Long-Term Actions

- Collect alternative credit data — utility payment history, rental records, mobile money transactions — to reduce reliance on CIBIL for thin-file applicants.
- Implement SHAP-based counterfactual explanations: 'Your loan would have been approved if your debt-to-income ratio were below 8x.'

- Annual third-party fairness audit by an independent AI ethics organisation.

11. Limitations and Future Improvements

11.1 Current Limitations

- Static Knowledge Base — the 12 rules and probability tables were defined manually and do not automatically update as lending patterns evolve.
- Naive Bayes Independence Assumption — the probabilistic layer assumes conditional independence between evidence signals. Income and debt ratio are correlated, which a full Bayesian network would capture.
- Dataset Size — 4,269 records is appropriate for an academic project but insufficient for production deployment, where hundreds of thousands of records spanning diverse economic conditions are the norm.
- Single Dataset Distribution — the dataset represents one specific population. Generalising across different countries or economic contexts requires retraining on local data.
- Fixed Fusion Weights — the 40/30/20/10 weight allocation was set heuristically. A meta-learner could learn optimal weights from data.
- No Temporal Modelling — each application is treated independently. An applicant's history of previous applications and decisions would be valuable context.

11.2 Future Improvements

- Gradient-Boosted Models — XGBoost or LightGBM as an additional ML layer, offering better generalization on larger datasets.
- Bayesian Network — replace Naive Bayes with a probabilistic graphical model (e.g., using pgmpy) that models feature dependencies.
- SHAP Integration — Shapley Additive Explanations for model-specific, individual-level feature attribution.
- REST API Deployment — wrap the inference engine in a FastAPI endpoint for integration into banking core systems.
- Real-Time Drift Monitoring — implement data drift detection to alert when the input distribution shifts from the training distribution.
- Database Backend — connect Streamlit to PostgreSQL for prediction storage, audit logging, and batch processing capability.

12. Conclusion

This project successfully demonstrates the design, implementation, and evaluation of a hybrid AI Expert System for loan approval decision support. The four-layer architecture — Rule-Based Expert System, Bayesian Probabilistic Reasoning, BFS Search, and Random Forest Machine Learning — provides complementary strengths that no single approach alone could match.

The system achieves 99.9% accuracy on the held-out test set, driven primarily by the high discriminative power of the engineered loan_to_income ratio (35.2% feature importance) and the

CIBIL score (26.8%). Every decision is accompanied by a complete, layered explanation covering all four reasoning paths — satisfying the explainability requirements central to responsible AI in financial services.

The ethics analysis reveals no significant approval-rate disparities by education or employment type, and only a modest and financially justified 3.1% below-average rate for the lowest income quintile. Accountability mechanisms including loan officer override authority, applicant appeal rights, and quarterly bias monitoring are built into the system design.

The Streamlit web application makes the system accessible to non-technical users, with interactive visualisations of all reasoning layers. The modular architecture supports future enhancements including SHAP explanations, Bayesian network reasoning, and production API deployment. This project demonstrates that explainable, fair, and accurate AI decision-making in finance is achievable through thoughtful integration of multiple AI paradigms.

References

Altman, E. I. (1968). Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *Journal of Finance*, 23(4), 589-609.

Baesens, B., Viaene, S., Van den Poel, D., Vanthienen, J., and Dedene, G. (2003). Bayesian neural network learning for repeat purchase modelling in direct marketing. *European Journal of Operational Research*, 138(1), 191-211.

Brachman, R. J. and Levesque, H. J. (2004). *Knowledge Representation and Reasoning*. Elsevier / Morgan Kaufmann. ISBN: 978-1558609327.

Chouldechova, A. (2017). Fair prediction with disparate impact: A study of bias in recidivism prediction instruments. *Big Data*, 5(2), 153-163.

Domingos, P. and Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning*, 29(2-3), 103-130.

Grewal, R. (2023). Loan Approval Dataset. Kaggle. Available at: <https://www.kaggle.com/datasets/rohitgrewal/loan-approval-dataset>

Hand, D. J. and Yu, K. (2001). Idiot's Bayes — not so stupid after all? *International Statistical Review*, 69(3), 385-398.

Khandani, A. E., Kim, A. J., and Lo, A. W. (2010). Consumer credit-risk models via machine-learning algorithms. *Journal of Banking and Finance*, 34(11), 2767-2787.

Lessmann, S., Baesens, B., Seow, H. V., and Thomas, L. C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247(1), 124-136.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. Available at: <https://scikit-learn.org>

Reserve Bank of India. (2022). Guidelines on Digital Lending. RBI/2022-23/111. Available at: <https://rbi.org.in>

Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. ISBN: 978-0134610993.

Streamlit Inc. (2023). Streamlit: The fastest way to build and share data apps. Available at: <https://streamlit.io>

Vedapradha, R., Hariharan, R., Suresh, P., and Divya, S. (2019). Artificial intelligence: An advanced prospect in banking sector. *Journal of Advanced Research in Dynamical and Control Systems*, 11(2), 1882-1886.

Assignment Title: Development of an AI-Based Expert System for Loan Approval Decision Support Using Probabilistic Reasoning and Machine Learning.

Total Marks: 100

Criteria	Weight (%)	Excellent (80–100)	Good (65–79)	Satisfactory (50–64)	Needs Improvement (35–49)	Poor (0–34)
Problem Analysis, Literature Review, Investigation	15	Deep understanding, strong investigation with research support	Good understanding, adequate investigation	Basic statement, limited research	Weak analysis, minimal research	No analysis or irrelevant problem context
Implementation of AI Techniques & Machine Learning Integration	25	Detailed design, Model selection, and Integration with the system	Good design, mostly correct coding	Adequate design, some errors	Incomplete or unclear design	No meaningful design or incorrect implementation
Dataset & Experimentation (K2, K3, K4)	25	Strong application of theory, Data preprocessing, and experimental setup clarity	Good application of ML Models and AI Techniques	Basic application, mostly descriptive	Weak link between theory and implementation	No evidence of theory applied
Evaluation, Result Analysis & Ethical Consideration	20	Clear results, Use of evaluation metrics, and Result analysis	Results with some analysis and limitations	Limited results, basic analysis	Weak or unclear results	No results or irrelevant analysis
Report Quality (Structure, Clarity, Presentation)	15	Well-organized, excellent formatting, clear diagrams	Organized with minor issues	Adequate structure, some missing sections	Poorly organized, unclear writing	Very disorganized or unreadable

KPA Justification Method:

Name of CEP (P's)	Addressed (Explain how it was addressed?) or —	
P1: Depth of Knowledge (Required K from K2, K3 & K4)	K2	Apply probabilistic reasoning to estimate the likelihood of diseases based on symptoms.
	K3	Implement AI search algorithms to find the best possible diagnosis path.
	K4	Use machine learning techniques to improve diagnostic performance.
P2: Conflicting Requirements	Balancing between cost efficiency and system performance.	
P3: Depth of Analysis	Analyze system performance on a variety of test cases and make improvements based on evaluation metrics such as accuracy, recall, and F1 score.	

Course Instructor Marks

Student ID: 23103259

Student Name: Rizwanul Kafi

Evaluator's Scoring Section:

Criteria	Weight (%)	Marks Awarded
Problem Analysis, Literature Review, Investigation	15	___/ 20
Implementation of AI Techniques & Machine Learning Integration	25	___/ 25
Dataset & Experimentation (K2, K3, K4)	25	___/ 20
Evaluation, Result Analysis & Ethical Consideration	20	___/ 20
Report Quality	15	___/ 15

Final Total___/ 100

Course Instructor Marks

Student ID: 23103274

Student Name: Isfaqur Rahman Seyam

Evaluator's Scoring Section:

Criteria	Weight (%)	Marks Awarded
Problem Analysis, Literature Review, Investigation	15	___/ 20
Implementation of AI Techniques & Machine Learning Integration	25	___/ 25
Dataset & Experimentation (K2, K3, K4)	25	___/ 20
Evaluation, Result Analysis & Ethical Consideration	20	___/ 20
Report Quality	15	___/ 15

Final Total_____/ 100